

Blending and Texture

Discussion #4

Written by:
Nithin

Topics

This section covers blending and textures.

Logistics

Remember Project 2 is due Friday, 9/29 at 11:59pm!

1 Taking it apart, and putting it together

We want to separate an image into details at different scales before blending it with another image. Let I be a 256×256 grayscale image. You may use the following operations:

- $\text{blur}(X)$: apply a normalized Gaussian low-pass filter.
- $\text{down}_2(X)$: keep every other row and column.
- $\text{expand}_2(X)$: upsample and interpolate to twice the height and width.

You may ignore boundary effects and numerical rounding.

(a) Set $G_0 = I$. Write expressions for the next two Gaussian pyramid levels, G_1 and G_2 , and give their sizes. Does the order of blurring and downsampling matter?

$$G_1 = \text{down}_2(\text{blur}(G_0)) \quad (128 \times 128), \quad G_2 = \text{down}_2(\text{blur}(G_1)) \quad (64 \times 64).$$

Blurring suppresses frequencies that the coarser grid cannot represent. Blurring after downsampling cannot undo aliasing that has already occurred.

(b) Construct Laplacian detail levels L_0 and L_1 from these Gaussian levels. Store $L_2 = G_2$ as the coarsest level. Write expressions for L_0 and L_1 . What does each of L_0, L_1, L_2 represent and what are their sizes?

$$L_0 = G_0 - \text{expand}_2(G_1), \quad L_1 = G_1 - \text{expand}_2(G_2), \quad L_2 = G_2.$$

L_0 and L_1 store detail lost at successive scales (approximately band-pass). L_2 stores the coarse, low-pass image. Their sizes are 256×256 , 128×128 , and 64×64 .

(c) Starting with L_2 , write the steps to recover I . Does the downsampling prevent exact reconstruction when all three levels are available? Why or why not?

$$\hat{G}_1 = L_1 + \text{expand}_2(L_2), \quad \hat{I} = L_0 + \text{expand}_2(\hat{G}_1).$$

The residuals store exactly the differences needed to recover the previous levels. Substitution gives

$\hat{G}_1 = G_1$ and $\hat{I} = I$, even though the Gaussian pyramid alone loses detail.

(d) Suppose Himanshu discards L_2 because it “just looks blurry.” Can he still recover the original image from L_0 and L_1 alone?

No. Consider two constant images of different brightness. They have identical $L_0 = L_1 = 0$ because the operations preserve constants, but have different L_2 .

2 Hiding the seam

Two different images A and B contain both broad, smooth color variations and fine texture. We want to blend the two images such that the left side comes from A and the right side from B .

Let M be a mask that is 1 on the left half and 0 on the right half. Ordinary feathering replaces this hard boundary with a smoothed mask $\tilde{M}$ transition:

$$C = \tilde{M} \odot A + (1 - \tilde{M}) \odot B,$$

where $\odot$ denotes pixel-wise multiplication.

(a) Consider two choices for the transition: a very narrow one and a very wide one. Which is more likely to leave an abrupt color seam? How about ghosted textures? Are there cases where we wouldn't want to use a wider transition?

A narrow transition can leave an abrupt change in broad color variations. A wide transition smooths that change, but averages both fine texture patterns over a larger region. When those patterns disagree, the result can look doubled or lose contrast. One transition width cannot generally handle both scales well.

(b) Suppose we separate each image into high frequency bands and a coarse, low-frequency image. Which should use a narrower transition and which should use a wider transition? Give your answer in the original image's coordinates.

Use narrower transitions for fine-detail bands, limiting the region where incompatible details are averaged. Use wider transitions for coarse content, spreading broad color changes smoothly. Successively coarser bands can use successively wider transitions in the original image's coordinates.

(c) Let L_k^A and L_k^B be the levels constructed in Problem 1. We also build a Gaussian pyramid M_k of the mask, with $M_0 = M$. Write an expression for the combined level L_k^C . How can we obtain the final image?

$$L_k^C = M_k \odot L_k^A + (1 - M_k) \odot L_k^B, \quad k = 0, 1, 2.$$

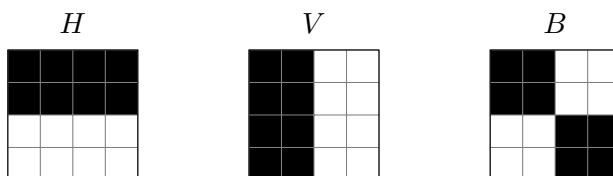
Blend the coarsest level as well, then reconstruct using the expansion and addition steps from Problem 1. A Gaussian mask supplies smooth weights between 0 and 1 at each scale. Its coarse transitions cover wider regions in the original image.

(d) Why did we use Gaussian levels for the mask rather than Laplacian levels of the mask?

Laplacian mask residuals instead describe changes between scales, can be negative, and do not encode the intended selection of A on the left and B on the right.

3 Same histogram, same texture?

H , V , and B , are all images where each square is one pixel: black is 0 and white is 1.



(a) For each image, compute its normalized pixel histogram $[p(0), p(1)]$. Can this histogram distinguish the three patterns? What information is lost?

Each image has eight black and eight white pixels, so all three histograms are $[1/2, 1/2]$. A pixel histogram counts intensities but discards their positions and which intensities occur next to one another.

(b) Instead, we can measure the differences between neighboring pixels:

$$D_x I[r, c] = I[r, c + 1] - I[r, c], \quad D_y I[r, c] = I[r + 1, c] - I[r, c].$$

Ignoring boundary effects (no wrapping; no padding) and *without* computing every entry, which patterns have nonzero D_x responses? Which have nonzero D_y responses?

D_x responds in V and B , which contain vertical edges. D_y responds in H and B , which contain horizontal edges. Thus H has only D_y responses, V only D_x , and B both.

(c) Summarize each image with two numbers:

$$f(I) = [\text{mean}(|D_x I|), \text{mean}(|D_y I|)].$$

Each difference array should have 12 entries. Ignoring boundary effects, compute $f(H)$, $f(V)$, and $f(B)$. Why can this descriptor distinguish patterns that the pixel histogram cannot? Give example patterns that f can not distinguish between.

Every responding direction has four entries of magnitude 1 and eight zeros. Therefore

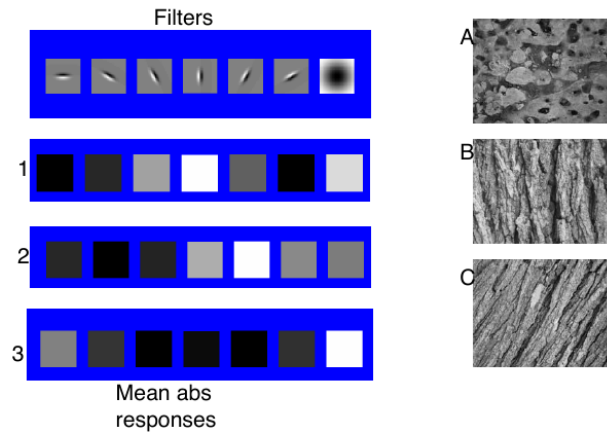
$$f(H) = [0, 1/3], \quad f(V) = [1/3, 0], \quad f(B) = [1/3, 1/3].$$

Filtering measures local spatial relationships before averaging. These statistics distinguish the three examples, but still discard the locations of the responses. Any pattern and its black–white inverse can not be disambiguated.

4 Filter responses as texture descriptors

Recall this lecture slide:

Can you match the texture to its histogram?



Each descriptor contains the *mean absolute response* to each filter.

(a) (*Warmup*) Match each texture to its descriptor.

A matches 3, B matches 1, and C matches 2.

(b) Why summarize the *absolute* filter responses rather than their signed means?

Opposite edge polarities cancel in a signed mean even when strong edges are present. Absolute values retain their strength.

(c) If we rotate a texture by 90° , should we expect its descriptor to remain unchanged? What descriptor can we use that would be more robust to rotations?

No. Rotation changes which oriented filters respond most strongly. Pooling responses across orientations reduces sensitivity, but loses the preferred direction that helped distinguish the textures.